
When Does Predictive Coding Produce a Policy Natural Gradient?

Parameter-Space NPG and Trust-Region PCPG

Anonymous Author(s)

Affiliation

Address

email

Abstract

Predictive coding (PC) has been interpreted as an adaptive trust-region method, but this does not imply that ordinary hidden-activity PC implements a natural policy gradient in parameter space. We isolate the missing condition: the inferred free variable must be a parameter displacement shared across rollout samples. For a diagonal-Gaussian policy, we construct a quadratic PC energy whose exact fixed point is the damped natural-gradient direction $(F_\theta + \lambda I)^{-1}g$. We derive an equivalent layer-local error-neuron system, implement the eliminated energy using Jacobian-vector products, analytic output precision, vector-Jacobian products, and conjugate-gradient inference, and add a trust-region variant that jointly infers a nonnegative precision β . No line search or update rejection is used. Synthetic tests give Fisher-equation cosine 0.99999994 and exact agreement between local and global perturbations. Four consecutive HalfCheetah updates place the local Fisher radius at 0.01 with inferred $\beta \in [3.45, 3.97]$; actual nonlinear KL is larger, 0.0438–0.0604. These results validate the local geometry and integration, not a nonlinear-KL guarantee or asymptotic control performance.

1 Introduction

Predictive coding networks infer latent neural activities by minimizing local prediction-error energies and then update synaptic parameters from settled errors. Innocenti et al. (Innocenti et al., 2023) show that this process can be understood as an adaptive trust-region method in neural-activity space. In policy optimization, however, “trust region” usually means a bound on policy KL, while “natural gradient” means preconditioning the policy gradient by the inverse parameter Fisher (Kakade, 2001; Schulman et al., 2015). These are different geometric claims.

Existing policy-PC constructions in this repository infer per-sample hidden activities or output targets and then consolidate them with an ordinary PC weight update. Even at activity convergence, their parameter update is generally a Jacobian-transpose mapping of an output displacement. It is not generally $(F_\theta + \lambda I)^{-1}g$. This distinction explains why converged tanh PCPG probes were not aligned with exact natural-gradient directions.

This paper makes four contributions. First, it gives sufficient assumptions under which PC inference exactly produces a damped parameter-space natural gradient. Second, it derives an equivalent layerwise error-neuron formulation with shared parameter perturbation activities. Third, it implements natural-gradient PCPG (NPG-PCPG) and trust-region PCPG (TR-PCPG) without materializing a Fisher matrix. Fourth, it validates the fixed-point equations and KL constraint synthetically and in short HalfCheetah integration runs.

34 2 Geometry of a Gaussian policy

35 Let a policy network produce $z_i(\theta) = (\mu_i, \ell_i)$ at state s_i , with $\ell_i = \log \sigma_i$. Let $\tilde{a}_i$ denote the
36 stored pre-tanh Gaussian action and A_i a fixed advantage estimate. Ignoring constants that cancel in
37 likelihood ratios,

$$\log \pi_\theta(\tilde{a}_i | s_i) = -\frac{1}{2} \sum_j \left[\frac{(\tilde{a}_{ij} - \mu_{ij})^2}{\sigma_{ij}^2} + 2\ell_{ij} \right]. \quad (1)$$

38 The advantage-weighted output score $u_i = A_i \nabla_{z_i} \log \pi_\theta$ has components

$$u_{\mu,i} = A_i (\tilde{a}_i - \mu_i) / \sigma_i^2, \quad (2)$$

$$u_{\ell,i} = A_i [((\tilde{a}_i - \mu_i) / \sigma_i)^2 - 1]. \quad (3)$$

39 The expected Fisher in output coordinates is

$$F_{\text{out},i} = \text{diag}(\sigma_i^{-2} I, 2I). \quad (4)$$

40 If log standard deviation is clipped, both the log-std score and its Fisher block are multiplied by the
41 local derivative mask of the clipping map.

42 Writing $J_i = \partial z_i / \partial \theta$, the empirical policy gradient and Fisher are

$$g = \frac{1}{N} \sum_i J_i^\top u_i, \quad F_\theta = \frac{1}{N} \sum_i J_i^\top F_{\text{out},i} J_i. \quad (5)$$

43 Locally, $N^{-1} \sum_i D_{\text{KL}}(\pi_\theta(\cdot | s_i) \| \pi_{\theta+d}(\cdot | s_i)) \approx \frac{1}{2} d^\top F_\theta d$. The output inverse $F_{\text{out},i}^{-1}$ alone there-
44 fore does not provide F_θ^{-1} for a shared neural policy.

45 3 Parameter-space predictive coding

46 3.1 Assumptions

47 **Assumption 1** (Fixed rollout). *States, pre-tanh actions, advantages, and old-policy outputs are held*
48 *fixed during one inference solve.*

49 **Assumption 2** (Local differentiability). *The policy output is differentiable at the current parameters.*
50 *For clipped outputs or piecewise-linear networks, the active derivative mask is fixed over the local*
51 *model.*

52 **Assumption 3** (Shared displacement). *The free PC variable $d \in \mathbb{R}^{|\theta|}$ is one parameter displacement*
53 *shared by all rollout samples, rather than one hidden activity per sample.*

54 **Assumption 4** (Regularization). $\lambda > 0$, or the zero-damping solution is defined by the Moore-
55 Penrose pseudoinverse of the generally singular empirical Fisher.

56 Define the natural output displacement $r_i = F_{\text{out},i}^{-1} u_i$ and the energy

$$E_{\text{NPG}}(d) = \frac{1}{2N} \sum_i \|J_i d - r_i\|_{F_{\text{out},i}}^2 + \frac{\lambda}{2} \|d\|^2. \quad (6)$$

57 **Theorem 1** (PC fixed point is a damped natural gradient). *Under Assumptions 1–4, the unique*
58 *minimizer of Eq. (6) is*

$$d^* = (F_\theta + \lambda I)^{-1} g. \quad (7)$$

59 At $\lambda = 0$, the minimum-norm solution is $d^* = F_\theta^+ g$ when g lies in the range of F_θ .

60 *Proof.* Expanding Eq. (6) and using $r_i = F_{\text{out},i}^{-1} u_i$ gives

$$E_{\text{NPG}}(d) = \frac{1}{2} d^\top F_\theta d - g^\top d + \frac{\lambda}{2} d^\top d + C. \quad (8)$$

61 Thus $\nabla_d E = (F_\theta + \lambda I)d - g$. Positive damping makes the Hessian positive definite, yielding
62 Eq. (7). The pseudoinverse statement follows from the minimum-norm solution of the undamped
63 normal equations. Full algebra appears in Appendix A. $\square$

64 Gradient-flow inference, $\tau \dot{d} = g - (F_\theta + \lambda I)d$, has the same fixed point. The implementation instead
65 applies conjugate gradients (CG) to this quadratic energy. CG is an accelerated inference procedure,
66 not a change to the objective.

67 3.2 Implicit Fisher computation

68 No Fisher matrix is formed. For a vector v , the exact empirical Fisher-vector product is evaluated as

$$v \xrightarrow{\text{JVP}} Jv \xrightarrow{F_{\text{out}}} F_{\text{out}}Jv \xrightarrow{\text{VJP}} J^\top F_{\text{out}}Jv = F_\theta v. \quad (9)$$

69 The method is therefore implicit in computation but explicitly Fisher-metric in mathematics.

70 4 Equivalent layerwise error neurons

71 The production solver eliminates intermediate perturbation activities. To expose a local PC interpretation, consider the JPC layer convention

$$h_l = W_l \phi_l(h_{l-1}) + b_l. \quad (10)$$

73 Introduce activity perturbations Δh_l and parameter activities $(\Delta W_l, \Delta b_l)$. Their linearized local
74 constraint is

$$c_l = \Delta h_l - W_l D_l \Delta h_{l-1} - \Delta W_l \phi_l(h_{l-1}) - \Delta b_l = 0, \quad (11)$$

75 where D_l is the activation Jacobian. Attach an error-neuron population q_l to each constraint and
76 define

$$\mathcal{L} = \frac{1}{N} \sum_i \left[\frac{1}{2} \Delta z_i^\top F_{\text{out},i} \Delta z_i - u_i^\top \Delta z_i + \sum_l q_{li}^\top c_{li} \right] + \frac{\lambda}{2} \|\Delta \theta\|^2. \quad (12)$$

77 Stationarity yields the local equations

$$q_{L,i} = u_i - F_{\text{out},i} \Delta z_i, \quad (13)$$

$$q_{l,i} = D_{l+1,i}^\top W_{l+1}^\top q_{l+1,i}, \quad (14)$$

$$\lambda \Delta W_l = \frac{1}{N} \sum_i q_{l,i} \phi_l(h_{l-1,i})^\top, \quad \lambda \Delta b_l = \frac{1}{N} \sum_i q_{l,i}. \quad (15)$$

78 These have the error-neuron form: an output mismatch drives q_L , neighboring weights and activation
79 derivatives transmit errors, and local Hebbian products drive shared parameter activities. Eliminating
80 Eq. (11) gives $\Delta z_i = J_i \Delta \theta$ and recovers Eq. (6). The implemented diagnostic checks both
81 representations, but production uses autodiff JVP/VJP and CG; it is not an asynchronous biologi-
82 cally local simulator.

83 5 NPG-PCPG and trust-region PCPG

84 5.1 Fixed-scale NPG-PCPG

85 NPG-PCPG infers d^* and directly applies $\theta' = \theta + \eta d^*$. For the reported HalfCheetah runs, $\eta = 0.05$.
86 There is no acceptance test. Adam or SGD is not applied afterward because an additional optimizer
87 would re-precondition the inferred direction.

88 5.2 Trust-region PCPG

89 The local quadratic trust-region subproblem is

$$\max_s g^\top s \quad \text{s.t.} \quad \frac{1}{2} s^\top F_\theta s \leq \delta. \quad (16)$$

90 Introduce a nonnegative precision neuron β and the saddle energy

$$E_{\text{TR}}(d, \beta) = -g^\top d + \beta \left(\frac{1}{2} d^\top F_\theta d - \epsilon \right), \quad \beta \geq 0. \quad (17)$$

91 The parameter neurons descend in d , while the precision neuron ascends in β . At an active-boundary
92 equilibrium,

$$\beta F_\theta d = g, \quad \frac{1}{2} d^\top F_\theta d = \epsilon. \quad (18)$$

93 Let $d_0 = F_\theta^+ g$ and $k_0 = \frac{1}{2} d_0^\top F_\theta d_0$. Solving Eq. (18) gives

$$\beta^* = \sqrt{k_0/\epsilon}, \quad d_{\text{TR}} = d_0/\beta^*. \quad (19)$$

Parameter-space PC update. (1) Feed forward all rollout states and retain old outputs. (2) Form g from fixed advantages and pre-tanh actions. (3) Infer d by CG using the implicit operator $v \mapsto F_\theta v + \lambda v$. (4a) NPG: apply the external scale η . (4b) TR: infer β so the local Fisher radius equals ϵ , then apply d/β directly.

Figure 1: Both algorithms share the same predictive-coding energy inference. They differ only in how the settled parameter activity is converted into a finite policy step.

94 An explicit local implementation may use projected dual ascent $\beta \leftarrow \max(0, \beta + \eta_\beta(R - \epsilon))$.
 95 Here η_β is only an inference-rate constant and does not change the settled solution. The production
 96 implementation computes β^* directly from Eq. (19); consequently, no η_β is present in the reported
 97 runs.

98 The resulting update is applied directly with no line search, clipping, or rejection. Closed-form old-
 99 to-new Gaussian KL is still logged as a diagnostic, but it does not affect the update. Thus TR-PCPG
 100 enforces the local Fisher radius, not the exact KL of the nonlinear post-update policy.

101 6 Experiments

102 6.1 Synthetic equivalence

103 We use a tanh JPC MLP with 128 samples, observation dimension 5, action dimension 2, width 8,
 104 depth 2, and $\lambda = 0.1$. Table 1 verifies the global normal equations, local error-neuron equations,
 105 and trust-region boundary.

Table 1: Synthetic fixed-point and equivalence checks on the supplied GPU.

Measurement	Result
Relative global KKT residual	2.293×10^{-4}
$\cos((F_\theta + \lambda I)d, g)$	0.999999940
Layerwise activity-constraint RMS	0
Maximum local/global output-JVP error	0
Relative layerwise parameter KKT residual	9.082×10^{-4}
NPG finite-step empirical KL	2.351×10^{-5}
TR local radius / actual KL	0.010000/1.281124

106 6.2 HalfCheetah integration

107 The integration run uses seed 1, 256 environments, rollout length 32, a tanh policy of width 64 and
 108 depth 2, normalized observations, rewards, and advantages. NPG uses $\lambda = 0.1$, 50 CG iterations,
 109 and $\eta = 0.05$. Direct TR uses $\lambda = 0$ and 1000 CG iterations because the undamped empirical
 110 Fisher is ill-conditioned. Four updates cover 32,768 environment steps. Table 2 reports geometry
 111 diagnostics, not final control performance.

Table 2: Ranges over four consecutive HalfCheetah updates.

Metric	NPG-PCPG	TR-PCPG
Fisher-equation cosine	0.99999964–1.00000000	0.99998391–0.99998969
Relative KKT residual	$2.58\text{--}3.20 \times 10^{-4}$	0.00471–0.00559
Local Fisher radius	$3.23\text{--}4.74 \times 10^{-5}$	0.010000
Actual full-batch KL	$3.276\text{--}4.789 \times 10^{-5}$	0.04383–0.06044
Inferred precision β	1	3.451–3.970
Applied scale	$\eta = 0.05$	$1/\beta = 0.252\text{--}0.290$

TR is applied directly. Actual KL is diagnostic and can exceed the local radius because the policy is nonlinear.

112 Both variants remain finite and improve the sampled surrogate without an acceptance rule. The
 113 Fisher-equation cosine stays near one. Direct TR-PCPG reaches the requested local radius on ev-

114 ery update, while actual KL is roughly four to six times larger. These short runs do not measure
115 asymptotic return, sample efficiency, or variance across seeds.

116 7 Related work

117 Natural policy gradient follows the Riemannian geometry induced by the policy Fisher (Kakade,
118 2001). TRPO solves a local KL-constrained problem using implicit Fisher-vector products and
119 CG (Schulman et al., 2015); PPO approximates a trust region through a clipped likelihood-ratio
120 objective (Schulman et al., 2017). Predictive coding has been related to backpropagation, Gauss–
121 Newton structure, and adaptive trust regions (Whittington and Bogacz, 2017; Innocenti et al., 2023).
122 The present construction should be read narrowly: it expresses an FVP/CG natural-gradient or TRPO
123 solve as inference in a PC energy over shared parameter perturbations. It does not show that ordinary
124 hidden-activity PC is automatically NPG.

125 8 Limitations and conclusion

126 The guarantee is local and empirical. It assumes a fixed rollout, fixed advantages, a differentiable
127 local policy model, and a diagonal Gaussian with network-emitted log standard deviation. ReLU
128 networks admit JVPs almost everywhere, but crossing activation kinks changes the local model
129 discontinuously; tanh is therefore used in the reported experiments. Damping stabilizes singular
130 empirical Fishers but changes the direction from the undamped NPG. Finite CG tolerance introduces
131 solve error, and the trust region is enforced only on sampled rollout states. The explicit error-neuron
132 equations are local, but the production implementation uses global autodiff operators and is not a
133 biologically local neural simulation.

134 The supplied GPU reports CUDA ptxas 12.4.131 and warns that CUDA versions through 12.6.2
135 can miscompile clamping edge cases. All checks pass, but experiments should be repeated on CUDA
136 12.6.3 or newer because the policy clips log standard deviation. Finally, four updates and one seed
137 are insufficient for performance claims; matched multi-seed comparisons against PPO and TRPO
138 remain necessary.

139 Subject to these limitations, the central result is exact: if the inferred PC state is a shared parameter
140 displacement and the output precision is the policy output Fisher, settled quadratic inference yields
141 the damped parameter-space natural gradient. Jointly inferring the nonnegative precision yields the
142 local quadratic trust-region boundary directly, without a line search or update rejection.

143 References

- 144 Innocenti, L., Singh, B., and Buckley, C. L. (2023). Understanding predictive coding as an adaptive
145 trust-region method. *arXiv preprint arXiv:2305.18188*.
- 146 Kakade, S. (2001). A natural policy gradient. *Advances in Neural Information Processing Systems*
147 *14*.
- 148 Schulman, J., Levine, S., Moritz, P., Jordan, M. I., and Abbeel, P. (2015). Trust region policy opti-
149 mization. *Proceedings of ICML*.
- 150 Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. (2017). Proximal policy opti-
151 mization algorithms. *arXiv preprint arXiv:1707.06347*.
- 152 Whittington, J. C. R. and Bogacz, R. (2017). An approximation of the error backpropagation algo-
153 rithm in a predictive coding network with local Hebbian synaptic plasticity. *Neural Computation*,
154 29(5):1229–1262.

A Detailed proof of the parameter-space result

For $\|x\|_M^2 = x^\top Mx$, substitute $r_i = F_{\text{out},i}^{-1}u_i$ into Eq. (6):

$$E(d) = \frac{1}{2N} \sum_i (J_i d - r_i)^\top F_{\text{out},i} (J_i d - r_i) + \frac{\lambda}{2} d^\top d \quad (20)$$

$$= \frac{1}{2N} \sum_i [d^\top J_i^\top F_{\text{out},i} J_i d - 2d^\top J_i^\top F_{\text{out},i} r_i + r_i^\top F_{\text{out},i} r_i] + \frac{\lambda}{2} d^\top d \quad (21)$$

$$= \frac{1}{2} d^\top \left(\frac{1}{N} \sum_i J_i^\top F_{\text{out},i} J_i \right) d - d^\top \left(\frac{1}{N} \sum_i J_i^\top u_i \right) + \frac{\lambda}{2} d^\top d + C \quad (22)$$

$$= \frac{1}{2} d^\top (F_\theta + \lambda I) d - g^\top d + C. \quad (23)$$

Differentiation gives $(F_\theta + \lambda I)d - g$. For $\lambda > 0$, the Hessian is positive definite because F_θ is positive semidefinite, so the fixed point is unique. At $\lambda = 0$, all minimizers satisfy $F_\theta d = g$; $F_\theta^+ g$ is the minimum-norm solution when the system is consistent.

B Initialization and local elimination

Every update begins with a feedforward initialization of all base activities h_l . Perturbation activities are initialized at zero and propagated through Eq. (11). Given a parameter tangent $\Delta\theta$, exact elimination recursively produces all Δh_l and hence $\Delta z_i = J_i \Delta\theta$. Conversely, the dual equations (15) propagate the output residual $u_i - F_{\text{out},i} \Delta z_i$ backward. Parameter stationarity combines these sample-local errors into the one shared $\Delta\theta$. The test module evaluates the primal constraints, global/local output-JVP agreement, and local parameter KKT residual.

C Implementation and reproduction

The production implementation is in `src/pc_algorithms/parameter_space_pc.py`. The explicit local diagnostic is `src/pc_algorithms/error_neuron_graph.py`; trainer integration is in `src/pc_algorithms/pc_actor_critic.py`. The configurations are `configs/mujoco_halfcheetah_pc_actor_critic_parameter_npg.yaml` and `..._parameter_tr.yaml`.

```
PYTHONPATH=src python scripts/test_parameter_space_pc.py
python scripts/run_train.py \
  --config configs/mujoco_halfcheetah_pc_actor_critic_parameter_npg.yaml
python scripts/run_train.py \
  --config configs/mujoco_halfcheetah_pc_actor_critic_parameter_tr.yaml
```

NPG scale η is controlled by `parameter_pc_step_size`; the ordinary policy optimizer learning rate is unused in parameter-PC modes. TR uses `parameter_pc_max_radius` and directly applies the inferred scale $1/\beta$; there are no line-search or backtracking settings. Value-PC retains the existing minibatch updates, while the policy performs one shared full-rollout parameter inference per rollout.

NeurIPS paper checklist

1. **Claims. Yes.** The abstract and Sections 1 and 8 limit the claims to geometry and integration checks and explicitly exclude control-performance claims.
2. **Limitations. Yes.** Section 8 discusses local linearization, damping, finite CG, sampled-state KL, policy class, biological locality, compute warnings, and absent multi-seed evaluation.

- 190 3. **Theory assumptions and proofs. Yes.** Assumptions 1–4 precede Theorem 1; the proof is given
191 in Section 3 and expanded in Appendix A.
- 192 4. **Experimental reproducibility. Yes.** Section 6 and Appendix C provide architecture, batch, seed,
193 damping, solver, environment, commands, and code locations.
- 194 5. **Open access to data and code. No.** The code exists in the working repository, but no public
195 archival URL or license is asserted in this manuscript.
- 196 6. **Experimental setting/details. Yes.** Section 6 specifies the synthetic and HalfCheetah settings
197 relevant to the reported diagnostics.
- 198 7. **Statistical significance. No.** The experiments are deterministic diagnostic and short integration
199 checks at one seed; no performance or population claim is made.
- 200 8. **Compute resources. No.** The supplied CUDA toolchain and update timing were observed, but
201 the GPU model and memory were not recorded.
- 202 9. **Code of ethics. Yes.** The work concerns optimization geometry and uses no human data, private
203 data, or deployed decision system.
- 204 10. **Broader impacts. N/A.** This is foundational optimizer analysis without a direct application or
205 deployed model.
- 206 11. **Safeguards. N/A.** No high-risk model or dataset is released.
- 207 12. **Licenses for existing assets. No.** The manuscript cites methods and environments but does not
208 yet audit all software-package licenses; this is required before submission.
- 209 13. **New assets. Yes.** New implementation modules and reproducibility commands are documented
210 in Appendix C; a public release and license remain future work.
- 211 14. **Crowdsourcing and human subjects. N/A.** No crowdsourcing or human-subject research is
212 involved.
- 213 15. **IRB approvals. N/A.** No human-subject research is involved.
- 214 16. **Declaration of LLM usage. N/A.** No LLM is part of the method or experiments; use for writing
215 or formatting does not affect the scientific procedure.